CHAPTER 8
ENCODERS, DECODERS, MULTIPLEXERS, AND DEMULTIPLEXERS

8.1 Introduction

Discrete quantities of information are represented in digital systems with binary codes. Sometimes binary codes are used for error detection during digital data transmission. A binary code of n bits is capable of representing up to 2^n distinct elements of the coded information. Discrete inputs to digital systems are encoded to binary-coded form by encoder circuits. In a reverse process, binary-coded data are decoded to discrete information by decoder circuits. Sometimes code conversion between one form to another is needed and is done by special types of encoder called code converter.

Multiplexing means transmitting a large number of information units over a smaller number of channels or lines. Demultiplexing is the reverse process of multiplexing. In demultiplexing process, information received on a smaller number of channels or lines is distributed over a large number of units. Multiplexer and demultiplexer circuits play an important role in digital systems where selection of path between multiple sources and single destination or between a single source and multiple destinations is required.

In this chapter design and application of various encoder, decoder, multiplexer, and demultiplexer circuits are studied.

8.2 Encoders

8.2.1 Design of Encoders

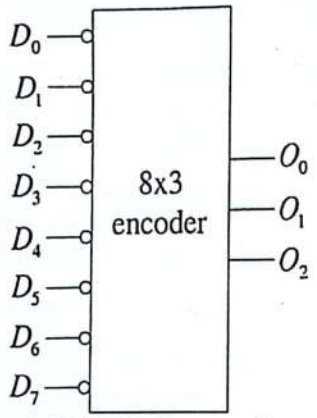
An *encoder* is a combinational circuit that has 2^n (or less) input lines and n output lines. Only one of the input lines is activated at a given time. The output lines generate the binary code corresponding to the activated input. The input lines can be activated in two ways – by setting the activated input to 0 and the other inputs to 1 (active-LOW input) or by setting the activated input to 1 and the other inputs to 0 (active-HIGH input). The block diagram and truth tables of an *octal-to-binary encoder* (also called *8-to-3-line encoder*) for both active-LOW and active-HIGH inputs are shown in Figure 8.1. From the truth tables, the Boolean function for the outputs can be written as below.

For active-LOW input

$$O_0 = D_1' + D_3' + D_5' + D_7' = (D_1 D_3 D_5 D_7)'$$
$$O_1 = D_2' + D_3' + D_6' + D_7' = (D_2 D_3 D_6 D_7)'$$
$$O_2 = D_4' + D_5' + D_6' + D_7' = (D_4 D_5 D_6 D_7)'$$

For active-HIGH input

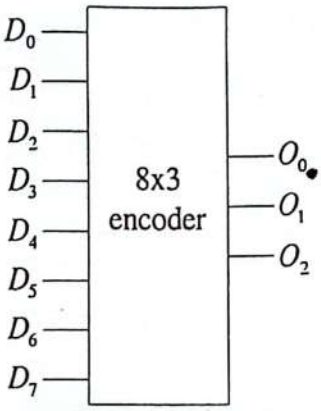
$$O_0 = D_1 + D_3 + D_5 + D_7$$
$$O_1 = D_2 + D_3 + D_6 + D_7$$
$$O_2 = D_4 + D_5 + D_6 + D_7$$



(a) Block diagram for active-LOW input

Inputs								Outputs		
D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0	O_2	O_1	O_0
1	1	1	1	1	1	1	0	0	0	0
1	1	1	1	1	1	0	1	0	0	1
1	1	1	1	1	0	1	1	0	1	0
1	1	1	1	0	1	1	1	0	1	1
1	1	1	0	1	1	1	1	1	0	0
1	1	0	1	1	1	1	1	1	0	1
1	0	1	1	1	1	1	1	1	1	0
0	1	1	1	1	1	1	1	1	1	1

(b) Truth table for active-LOW input



(c) Block diagram for active-HIGH input

Inputs								Outputs		
D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0	O_2	O_1	O_0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

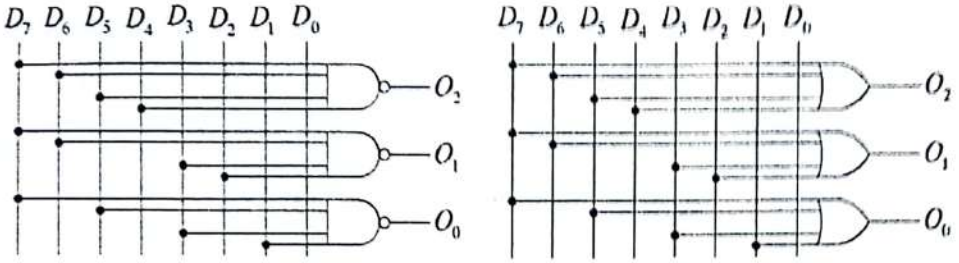
(d) Truth table for active-HIGH input

Figure 8.1 Block diagram and truth table of octal-to-binary encoder (8-to-3-line encoder).

The logic implementation of the octal-to-binary encoder for both active-LOW and active-HIGH inputs is shown in Figure 8.2. Note that D_0 input is not connected to gates.

The encoder of Figure 8.2 assumes that only one input line can be activated at any time; otherwise the circuit has no meaning. Note that the circuit has eight inputs and could have $2^8 = 256$ possible input combinations. Only eight of these combinations have any meaning. The other input combinations are don't care

conditions. Encoders of this type are not available in IC package, since they can be easily constructed with gates. The type of encoders available in IC form is called a *priority encoder*. These encoders establish an input priority to ensure that only the highest-priority input line is encoded. Thus, in truth tables of Figure 8.1, if priority is given to an input with higher subscript number over one with a lower subscript number, then if both D_2 and D_5 are activated simultaneously, the output will be 101 because D_5 has a higher priority over D_2. Of course, the truth table of a priority encoder is different from the one of Figure 8.1.



(a) Active-LOW input (b) Active-HIGH input

Figure 8.2 Logic implementation of octal-to-binary encoder (8-to-3-line encoder).

8.2.2 Available MSI Encoder ICs

Available MSI priority encoder ICs are given in Table 8.1.

Table 8.1 Available MSI encoder ICs.

IC No.	Description
74147	Decimal-to-BCD priority encoder with active-LOW input and inverted outputs.
74148	Octal-to-binary priority encoder with active-LOW input and inverted outputs.

IC 74147 is introduced here. The pin assignment of IC 74147 is shown in Figure 8.3. IC 74147 9-input priority encoder accepts data from nine active-LOW inputs (D_1' - D_9') and provides a binary representation on the four active-LOW outputs (O_0' - O_3'). A priority is assigned to each input so that when two or more inputs are simultaneously active, the input with the highest priority is represented on the output, with input line D_9' having the highest priority. The IC provides the 10-to-4-line priority encoding function by use of the implied decimal "zero". The "zero" is encoded when all nine data inputs are HIGH, forcing all four outputs HIGH.

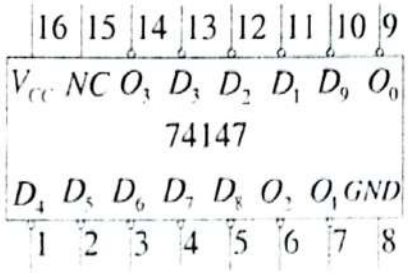


Figure 8.3 IC type 74147 Decimal-to-BCD priority encoder

8.3 Decoders

8.3.1 Design of Decoders

A *decoder* is a combinational circuit that accepts n inputs that represent a binary code and activate only the output that corresponds to the input code. The number of output lines may be a maximum of 2^n. If the n-bit decoded information has unused or don't care combinations, the decoder output will have less than 2^n outputs. The output lines can be activated in two ways – by setting the activated output to 0 and the other outputs to 1 (active-LOW output) or by setting the activated output to 1 and the other outputs to 0 (active-HIGH output). If the number of output lines is less than 2^n, the decoder is often designed such that all the outputs remain inactivated when an invalid input combination (unused code) is applied.

Binary-to-octal decoder

The block diagrams and truth tables of a *binary-to-octal decoder* (also called *3-to-8-line decoder* or *1-of-8 decoder*) for both active-LOW and active-HIGH outputs are shown in Figure 8.4. From the truth tables, the Boolean functions for the outputs can be written as below.

For active-LOW output

$$O_0 = A_2 + A_1 + A_0 = (A_2' A_1' A_0')'$$
$$O_1 = A_2 + A_1 + A_0' = (A_2' A_1' A_0)'$$
$$O_2 = A_2 + A_1' + A_0 = (A_2' A_1 A_0')'$$
$$O_3 = A_2 + A_1' + A_0' = (A_2' A_1 A_0)'$$
$$O_4 = A_2' + A_1 + A_0 = (A_2 A_1' A_0')'$$
$$O_5 = A_2' + A_1 + A_0' = (A_2 A_1' A_0)'$$
$$O_6 = A_2' + A_1' + A_0 = (A_2 A_1 A_0')'$$
$$O_7 = A_2' + A_1' + A_0' = (A_2 A_1 A_0)'$$

For active-HIGH output

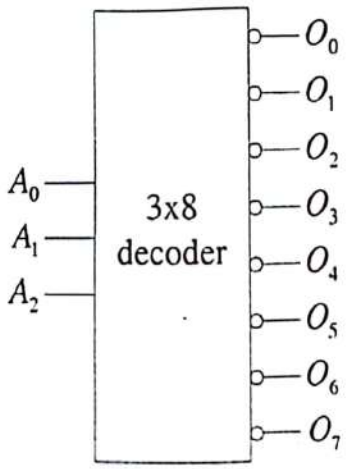
$$O_0 = A_2' A_1' A_0'$$
$$O_1 = A_2' A_1' A_0$$
$$O_2 = A_2' A_1 A_0'$$
$$O_3 = A_2' A_1 A_0$$
$$O_4 = A_2 A_1' A_0'$$

$$O_5 = A_2 A_1' A_0$$

$$O_6 = A_2 A_1 A_0'$$

$$O_7 = A_2 A_1 A_0$$

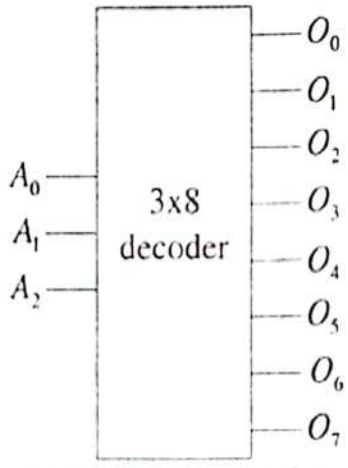
The logic implementation of the binary-to-octal decoder for both active-LOW and active-HIGH outputs is shown in Figure 8.5. Note that the outputs of the decoder with active-HIGH output are simply inverted to obtain the decoder with active-LOW output.



(a) Block diagram for active-LOW output

Inputs			Outputs							
A_2	A_1	A_0	O_7	O_6	O_5	O_4	O_3	O_2	O_1	O_0
0	0	0	1	1	1	1	1	1	1	0
0	0	1	1	1	1	1	1	1	0	1
0	1	0	1	1	1	1	1	0	1	1
0	1	1	1	1	1	1	0	1	1	1
1	0	0	1	1	1	0	1	1	1	1
1	0	1	1	1	0	1	1	1	1	1
1	1	0	1	0	1	1	1	1	1	1
1	1	1	0	1	1	1	1	1	1	1

(b) Truth table for active-LOW output



(c) Block diagram for active-HIGH output

Inputs			Outputs							
A_2	A_1	A_0	O_7	O_6	O_5	O_4	O_3	O_2	O_1	O_0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

(d) Truth table for active-HIGH output

Figure 8.4 Block diagram and truth table of binary-to-octal decoder (3-to-8-line decoder or 1-of-8 decoder).

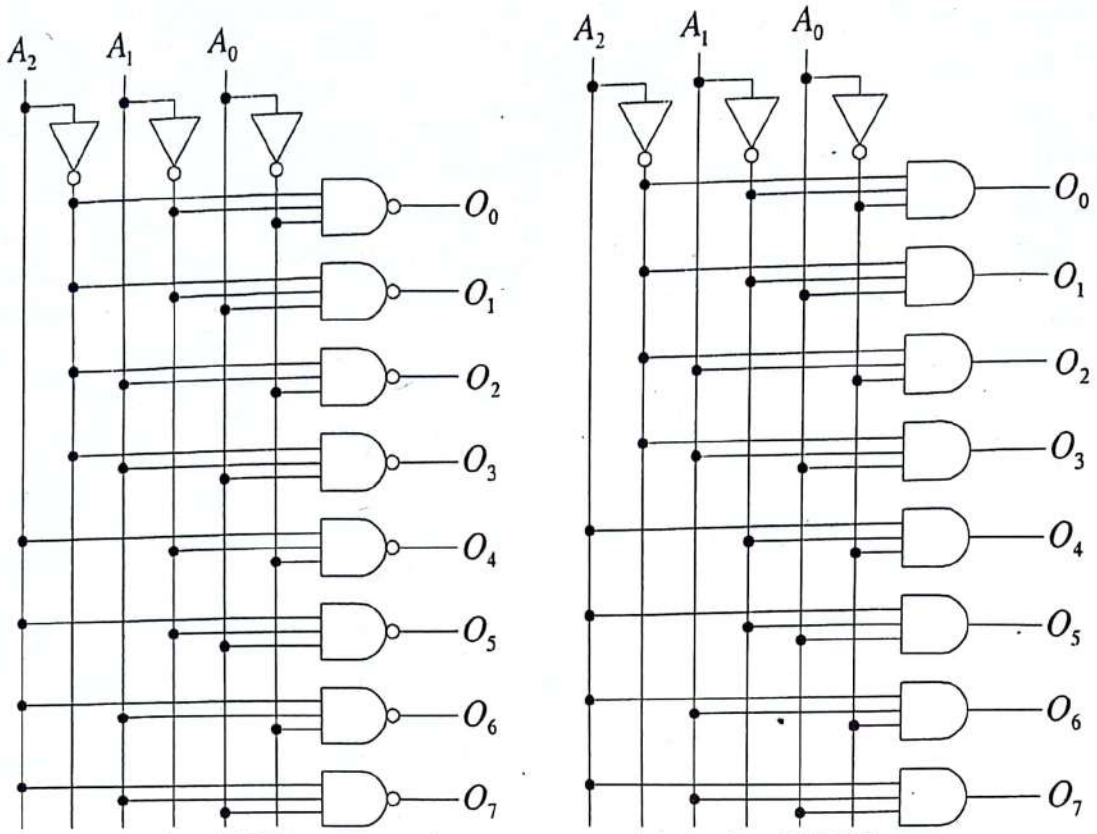


Figure 8.5 Logic implementation of binary-to-octal decoder (3-to-8-line decoder or 1-of-8 decoder).

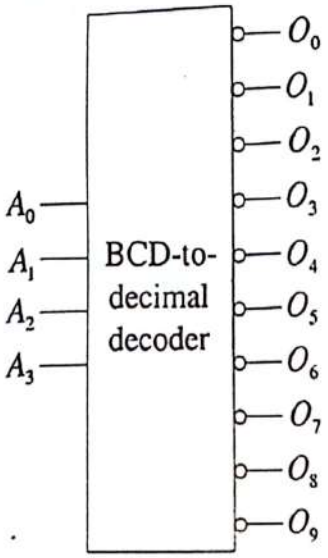
The decoders with binary inputs are called n-to-m line decoder (or 1-of-m decoders) where $m \leq 2^n$. Their purpose is to generate the 2^n (or less) minterms (active-HIGH output) or maxterms (active-LOW output) of n input variables.

BCD-to-decimal decoder

Sometimes decoders are designed to accept coded binary number such as BCD number or Excess-3 number. The design of a BCD-to-decimal decoder with active-LOW output is given here. The block diagram and truth table of a BCD-to-decimal decoder with active-LOW output are given in Figure 8.6. Note that for the invalid BCD inputs (1010 to 1111) no output line is activated. The Boolean functions for the outputs can be written from the truth table as

$$O_0 = A_3 + A_2 + A_1 + A_0 = (A_3' A_2' A_1' A_0')'$$
$$O_1 = A_3 + A_2 + A_1 + A_0' = (A_3' A_2' A_1' A_0)'$$
$$O_2 = A_3 + A_2 + A_1' + A_0 = (A_3' A_2' A_1 A_0')'$$
$$O_3 = A_3 + A_2 + A_1' + A_0' = (A_3' A_2' A_1 A_0)'$$
$$O_4 = A_3 + A_2' + A_1 + A_0 = (A_3' A_2 A_1' A_0')'$$
$$O_5 = A_3 + A_2' + A_1 + A_0' = (A_3' A_2 A_1' A_0)'$$

$$O_6 = A_3 + A_2' + A_1' + A_0 = (A_3' A_2 A_1 A_0')'$$
$$O_7 = A_3 + A_2' + A_1' + A_0' = (A_3' A_2 A_1 A_0)'$$
$$O_8 = A_3' + A_2 + A_1 + A_0 = (A_3 A_2' A_1' A_0')'$$
$$O_9 = A_3' + A_2 + A_1 + A_0' = (A_3 A_2' A_1' A_0)'$$



(a) Block diagram

Inputs				Outputs									
A_3	A_2	A_1	A_0	O_9	O_8	O_7	O_6	O_5	O_4	O_3	O_2	O_1	O_0
0	0	0	0	1	1	1	1	1	1	1	1	1	0
0	0	0	1	1	1	1	1	1	1	1	1	0	1
0	0	1	0	1	1	1	1	1	1	1	0	1	1
0	0	1	1	1	1	1	1	1	1	0	1	1	1
0	1	0	0	1	1	1	1	1	0	1	1	1	1
0	1	0	1	1	1	1	1	0	1	1	1	1	1
0	1	1	0	1	1	1	0	1	1	1	1	1	1
0	1	1	1	1	1	0	1	1	1	1	1	1	1
1	0	0	0	1	0	1	1	1	1	1	1	1	1
1	0	0	1	0	1	1	1	1	1	1	1	1	1
1	0	1	0	1	1	1	1	1	1	1	1	1	1
1	0	1	1	1	1	1	1	1	1	1	1	1	1
1	1	0	0	1	1	1	1	1	1	1	1	1	1
1	1	0	1	1	1	1	1	1	1	1	1	1	1
1	1	1	0	1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1

(b) Truth table

Figure 8.6 Block diagram and truth table of BCD-to-decimal decoder with active-LOW output.

The logic implementation of the BCD-to-decimal decoder with active-LOW output is shown in Figure 8.7.

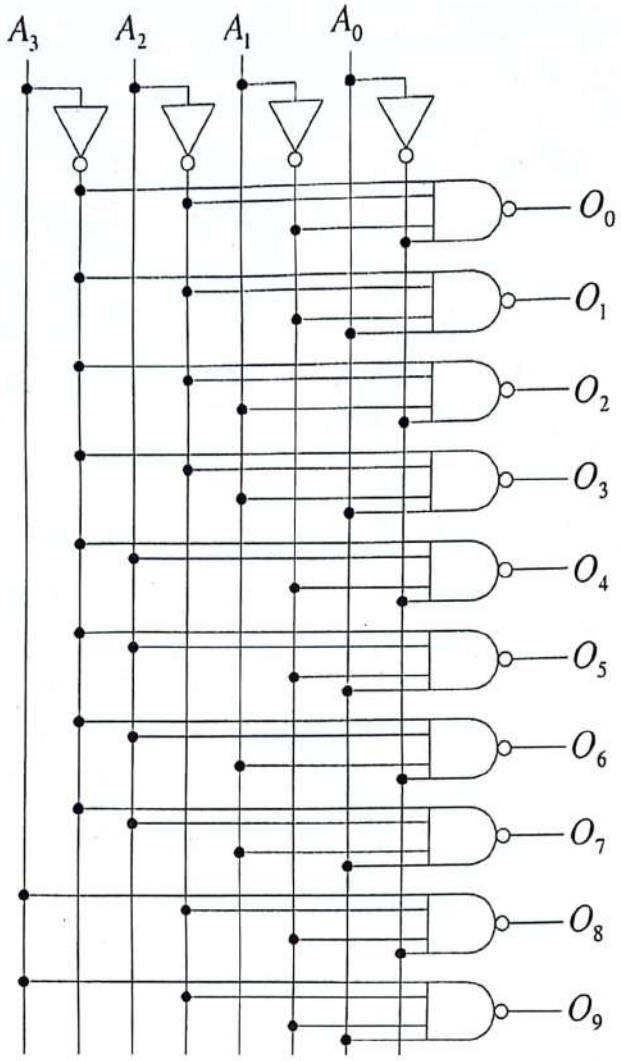


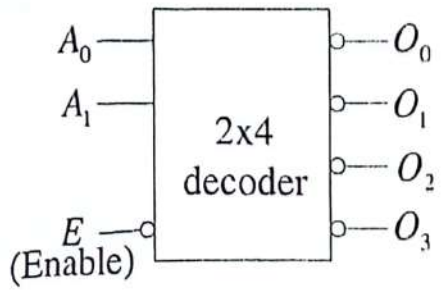
Figure 8.7 Logic implementation of a BCD-to-decimal decoder with active-LOW output.

8.3.2 Decoders with Enable Input

Some decoders have enable input that is used to control the operation of the decoder. If the enable input is active, the output corresponding to the input is activated. If the enable input is not active, no output is activated. The block diagram and truth table of a 2-to-4-line decoder with active-LOW output and active-LOW enable is shown in Figure 8.8. From truth table the Boolean functions for the outputs can be written as

$$O_0 = E + A_1 + A_0 = (E'A_1'A_0')'$$
$$O_1 = E + A_1 + A_0' = (E'A_1'A_0)'$$
$$O_2 = E + A_1' + A_0 = (E'A_1A_0')'$$
$$O_3 = E + A_1' + A_0' = (E'A_1A_0)'$$

The logic implementation of the 2-to-4-line decoder with active-LOW output and active-LOW enable input is shown in Figure 8.9.



(a) Block diagram

Inputs			Outputs			
E	A_1	A_0	O_3	O_2	O_1	O_0
0	0	0	1	1	1	0
0	0	1	1	1	0	1
0	1	0	1	0	1	1
0	1	1	0	1	1	1
1	0	0	1	1	1	1
1	0	1	1	1	1	1
1	1	0	1	1	1	1
1	1	1	1	1	1	1

(b) Truth table

Figure 8.8 Block diagram and truth table of 2-to-4-line decoder with active-LOW output and active-LOW enable input.

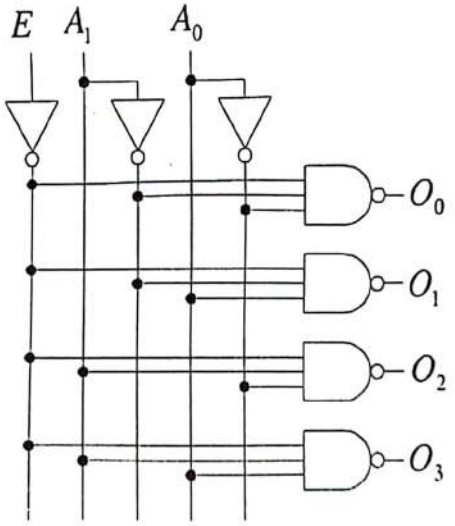


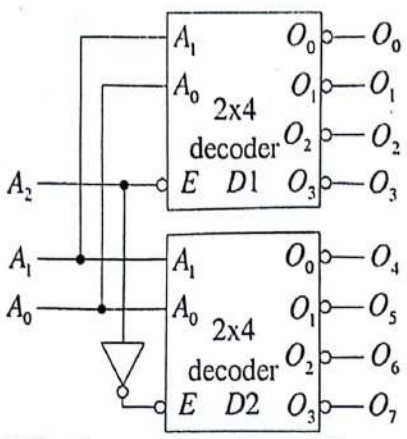
Figure 8.9 Logic implementation of 2-to-4-line decoder with active-LOW output and active-LOW enable input.

8.3.3 Expansion of Decoder

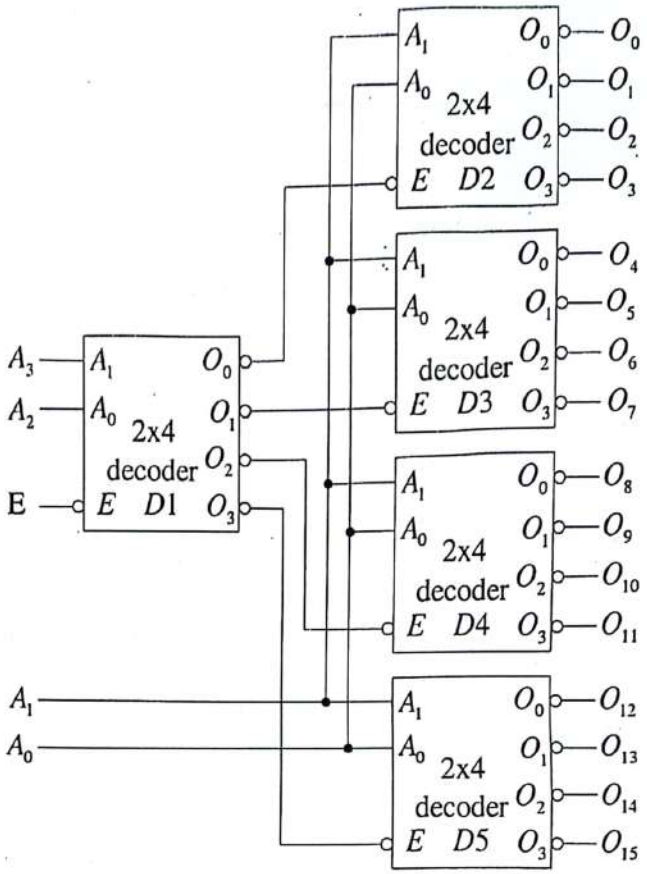
To meet the larger inputs needs, decoders can be expanded by using *enable* input line. Figure 8.10(a) shows a 3-to-8-line decoder and Figure 8.10(b) shows a 4-to-16-line decoder using 2-to-4-line decoders.

In Figure 8.10(a), when $A_2 = 0$, decoder $D1$ is enabled and decoder $D2$ is disabled. The output corresponding to A_1A_0 input of the decoder $D1$ is activated. When $A_2 = 1$, decoder $D2$ is enabled and decoder $D1$ is disabled. The output corresponding to A_1A_0 input of the decoder $D2$ is activated. For example, if $A_2A_1A_0 = 101$, decoder $D2$ will be enabled since $A_2 = 1$ and the output O_5 of

the decoder $D2$ will be activated since $A_1A_0 = 01$. Therefore, output O_5 of the combined decoder will be activated.



(a) Implementation of 3-to-8-line decoder with 2-to-4-line decoders



(b) Implementation of 4-to-16-line decoder with 2-to-4-line decoders

Figure 8.10 Expansion of decoder.

In Figure 8.10(b), when high-order bits $A_3A_2 = 00$, the output O_0 of the decoder $D1$ will be LOW and the decoder $D2$ will be enabled. The output of the decoder $D2$ corresponding to the low-order bits A_1A_0 will be activated. Similarly, when $A_3A_2 = 01$, the decoder $D3$ will be enabled and the output of the decoder $D3$ corresponding to A_1A_0 will be activated. When $A_3A_2 = 10$, decoder $D4$ will be enabled and the output of the decoder $D4$ corresponding to A_1A_0 will be activated. Finally, when $A_3A_2 = 11$, decoder $D5$ will be enabled and the output of the decoder $D5$ corresponding to A_1A_0 will be activated. For example, when $A_3A_2A_1A_0 = 0110$, that is high-order bits $A_3A_2 = 01$ and low-order bits $A_1A_0 = 10$, the O_1 output of the decoder $D1$ will be LOW, which will enable the decoder $D3$. Then the output O_2 of the decoder $D3$ will be activated, which is the output O_6 of the combined decoder.

8.3.4 Combinational Logic Implementation with Decoder

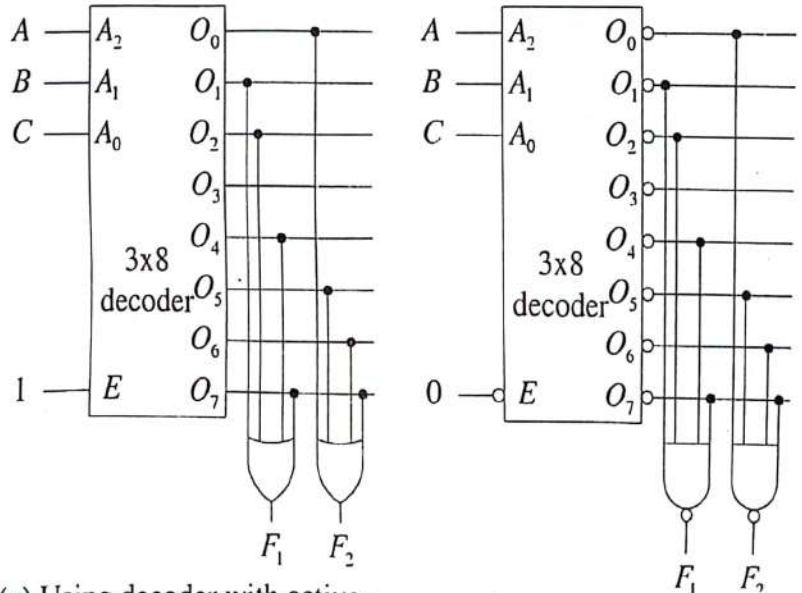
A decoder provides the 2^n minterms (active-HIGH output) or maxterms (active-LOW output) of n input variables. Since, any Boolean function can be expressed as sum of minterms (canonical sum of products) or product of maxterms (canonical product of sums), decoders can be used to implement Boolean function in conjunction with some external gates. In this implementation method, any combinational circuit with n inputs and m outputs can be implemented with an n- to-2^n-line decoder and m OR/NAND or AND/NOR gates.

Implementation of Canonical Sum of Products Function

For implementing an n-variable canonical sum of products function, the n inputs of the function are applied to the n inputs of an n- to-2^n-line decoder with active-HIGH outputs. As the outputs of the decoder generate the minterms of an n-variable function, the outputs of the decoder corresponding to the minterms of the function are ORed with an OR gate to form the sum. Implementation of the following 3-variable functions using a 3-to-8-line decoder with active-HIGH output is shown in Figure 8.11(a).

$$F_1(A,B,C) = \sum(1,2,4,7)$$
$$F_2(A,B,C) = \sum(0,5,6,7)$$

The canonical sum of products function can also be implemented using decoder with active-LOW output. In this decoder, the outputs represent the complement of the minterms. In this implementation, the complemented minterms are combined with a NAND gate to form the sum of minterms. The procedure is obvious from DeMorgan's theorem. Implementation of the above functions using a 3-to-8-line decoder with active-LOW output is shown in Figure 8.11(b).



(a) Using decoder with active-HIGH outpt

(a) Using decoder with active-LOW outpt

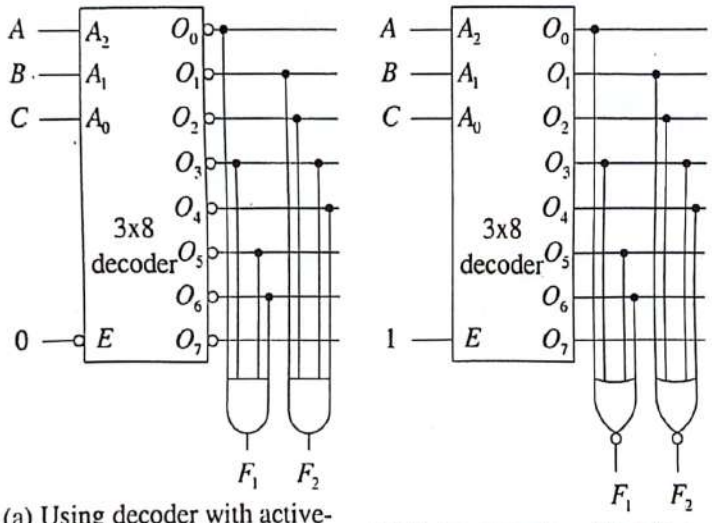
Figure 8.11 Implementation of canonical sum of product functions $F_1(A,B,C) = \sum(1,2,4,7)$ and $F_2(A,B,C) = \sum(0,5,6,7)$ using decoders.

Implementation of Canonical Product of Sums Function

For implementing an n-variable canonical product of sums function, the n inputs of the function are applied to the n inputs of an n-to-2^n-line decoder with active-LOW outputs. As the outputs of the decoder generate the maxterms of an n-variable function, the outputs of the decoder corresponding to the maxterms of the function are ANDed with an AND gate to form the product. Implementation of the following 3-variable functions using a 3-to-8-line decoder with active-LOW output is shown in Figure 8.12(a).

$$F_1(A,B,C) = \prod(0,3,5,6)$$
$$F_2(A,B,C) = \prod(1,2,3,4)$$

The canonical product of sums function can also be implemented using decoders with active-HIGH outputs. In this decoder, the outputs represent the complement of the maxterms. In this implementation, the complemented maxterms are combined with a NOR gate to form the product of maxterms. The procedure is obvious from DeMorgan's theorem. Implementation of the above functions using a 3-to-8-line decoder with active-HIGH output is shown in Figure 8.12(b).



(a) Using decoder with active-LOW outpt

(a) Using decoder with active-HIGH outpt

Figure 10.12 Implementation of canonical product of sums function $F_1(A,B,C) = \prod(0,3,5,6)$ and $F_2(A,B,C) = \prod(1,2,3,4)$ using decoders.

If the number of minterms in a function is less than $2^n/2$, then the implementation of Figure 8.11 is suitable, otherwise the implementation of Figure 8.12 is suitable.

The decoder method can be used to implement any combinational circuit. However, its implementation must be compared with all other possible implementations to determine the best solution. In some cases this method may provide the best implementation, especially if the combinational circuit has many outputs and if each output function is expressed with a small number of minterms or maxterms.

8.3.5 Available MSI Decoder ICs

List of available MSI decoder ICs are given in Table 8.2.

Table 8.2 Available MSI decoder ICs.

IC No.	Description
74139	Dual 2-to-4-line decoder with active-LOW output
74155	Dual 2-to-4-line decoder with active-LOW output
74156	Dual 2-to-4-line decoder with active-LOW output (open-collector)
74138	3-to-8-line decoder with active-LOW output
74154	4-to-16-line decoder with active-HIGH output
7442	BCD-to-decimal decoder with active-LOW output.

IC 74138 is introduced here. The pin assignment of IC 74138 is shown in Figure 8.13. A_0 through A_2 are three address lines and O_0' through O_7' are eight active-LOW outputs. It has two active-LOW (E_1' and E_2') and one active-HIGH (E_3) enable inputs internally ANDed to produce a single enable signal.

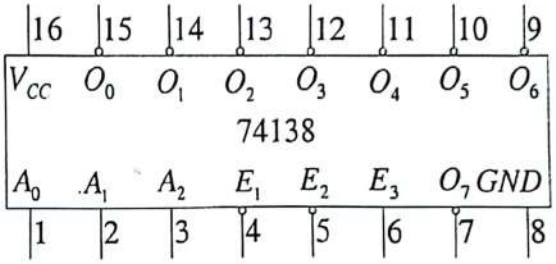


Figure 8.13 IC type 74138 3-to-8-line decoder with active-LOW output.

8.4 Code Converters

There are a wide variety of binary codes used in digital systems. Some of these codes are BCD, Excess-3, etc. Different digital systems use different codes. It is sometimes necessary to use the output of one system as the input to another. Therefore, it is required to convert from one code to another. This sort of code converters are combinational circuits and may be regarded as special type of encoders, which encodes from one form to another. Code converters can be designed using classical method of combinational logic design or using MSI circuits. Design of BCD-to-Excess-3 code converter is given here.

The block diagram and truth table for BCD-to-Excess-3 code converter is shown in Figure 8.14. Note that the input combinations 1010 to 1111 are invalid BCD code and will not appear in the input. Therefore, their outputs are don't care states. The output functions are simplified using K-maps in Figure 8.15, which gives

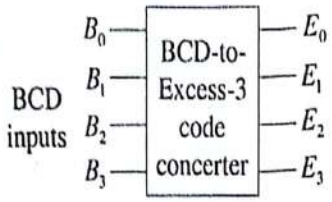
$$E_0 = B_0'$$

$$E_1 = B_1'B_0' + B_1B_0$$

$$E_2 = B_2'B_0 + B_2'B_1 + B_2B_1'B_0'$$

$$E_3 = B_3 + B_2B_0 + B_2B_1$$

The logic implementation for BCD-to-Excess-3 code converter is shown in Figure 8.16.

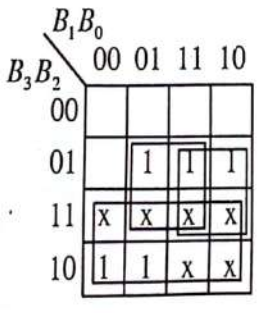


(a) Block diagram

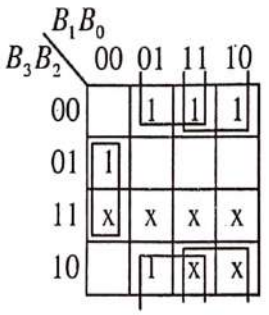
Input BCD				Output Excess-3			
B_3	B_2	B_1	B_0	E_3	E_2	E_1	E_0
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0
1	0	1	0	x	x	x	x
1	0	1	1	x	x	x	x
1	1	0	0	x	x	x	x
1	1	0	1	x	x	x	x
1	1	1	0	x	x	x	x
1	1	1	1	x	x	x	x

(b) Truth table

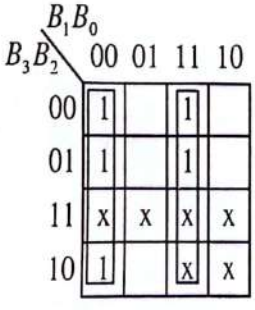
Figure 8.14 Block diagram and truth table for BCD-to-Excess-3 code converter.



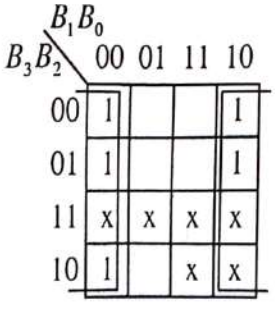
(a) $E_3 = B_3 + B_2B_0 + B_2B_1$



(b) $E_2 = B_2'B_0 + B_2'B_1 + B_2B_1'B_0'$



(c) $E_1 = B_1'B_0' + B_1B_0$



(d) $E_0 = B_0'$

Figure 8.15 K-maps for BCD-to-Excess-3 code converter.

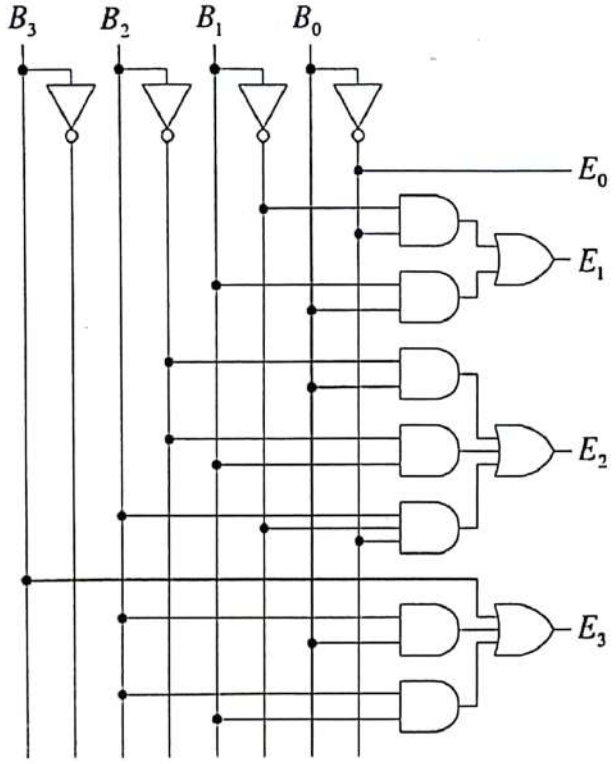


Figure 8.16 Logic implementation for BCD-to-Excess-3 code converter.

The BCD-to-Excess-3 code converter can be implemented using a 4-bit binary adder. From the truth table of Figure 8.14(b), it can be noted that Excess-3 code can be obtained by adding binary 3 (0011) with the BCD code. This addition can be implemented by a 4-bit binary adder as shown in Figure 8.17.

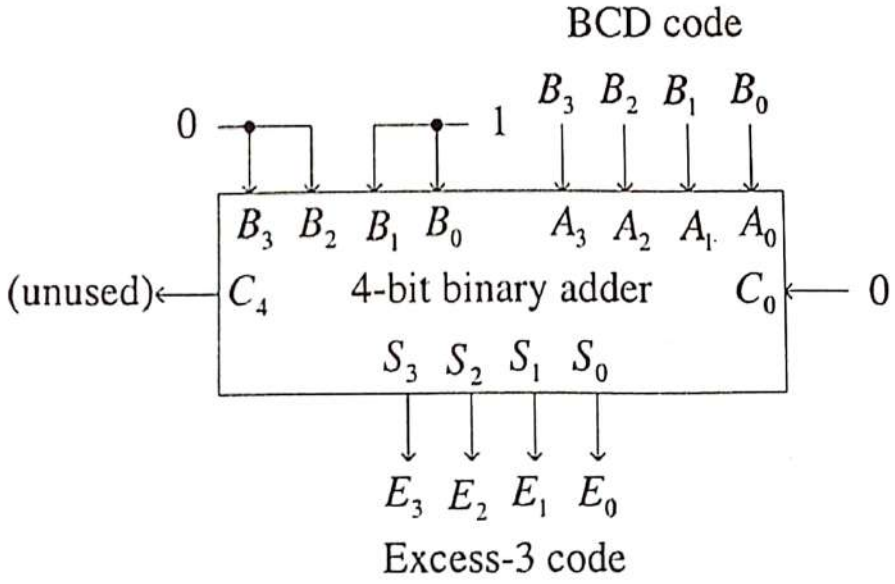


Figure 8.17 BCD-to-Excess-3 code converter using 4-bit binary adder.

8.5 BCD-to-7-Segment Display Decoder

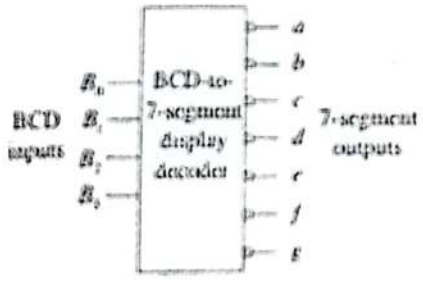
8.5.1 Design of BCD-to-7-Segment Display Decoder

In many digital systems, we prefer to see the output in a decimal format. Seven segment display is the most popular display device used in digital systems. For displaying data using this device, the data have to be converted from BCD to 7-segment code.

In 7-segment display seven light-emitting diodes (LEDs) are arranged as in Figure 8.18(a). By controlling the current through each LED, some segments will be light and others will be dark so that the desired digit pattern will be generated. Figure 8.18(b) shows the segment patterns for digits 0 to 9. For example, to display a "6", the segments *a*, *c*, *d*, *e*, *f*, *g* are made bright while segment *b* is dark. A BCD-to-7-segment display decoder is used to take a 4-bit BCD input and to activate the outputs that will pass current through the appropriate segments to display the decimal digit. The block diagram and truth table of a BCD-to-7-segment decoder with active-LOW outputs is shown in Figure 8.19. Note that for the invalid BCD inputs (1010 to 1111) no segment will be activated.



Figure 8.18 (a) 7-segment arrangement and (b) activated segments for digits 0 to 9.



(a) Block diagram

BCD inputs				7-segment outputs						
B_3	B_2	B_1	B_0	*a*	*b*	*c*	*d*	*e*	*f*	*g*
0	0	0	0	0	0	0	0	0	0	1
0	0	0	1	1	0	0	1	1	1	1
0	0	1	0	0	0	1	0	0	1	0
0	0	1	1	0	0	0	0	1	1	0
0	1	0	0	1	0	0	1	1	0	0
0	1	0	1	0	1	0	0	1	0	0
0	1	1	0	0	1	0	0	0	0	0
0	1	1	1	0	0	0	1	1	1	1
1	0	0	0	0	0	0	0	0	0	0
1	0	0	1	0	0	0	0	1	0	0
1	0	1	0	1	1	1	1	1	1	1
1	0	1	1	1	1	1	1	1	1	1
1	1	0	0	1	1	1	1	1	1	1
1	1	0	1	1	1	1	1	1	1	1
1	1	1	0	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1	1

(b) Truth table

Figure 8.19 Block diagram and truth table of BCD-to-7-segment display decoder with active-LOW output

The output functions are simplified using K-maps in Figure 8.20, which gives

$$a = B_3B_2 + B_3B_1 + B_2B_1'B_0' + B_3'B_2'B_1'B_0$$
$$b = B_3B_2 + B_3B_1 + B_2B_1'B_0 + B_2B_1B_0'$$
$$c = B_3B_2 + B_3B_1 + B_2'B_1B_0'$$
$$d = B_3B_2 + B_3B_1 + B_2B_1'B_0' + B_2B_1B_0 + B_3'B_2'B_1'B_0$$
$$e = B_3B_1 + B_0 + B_2B_1'$$
$$f = B_3B_2 + B_1B_0 + B_2'B_1 + B_3'B_2'B_0$$
$$g = B_3B_2 + B_3B_1 + B_2B_1B_0 + B_3'B_2'B_1'$$

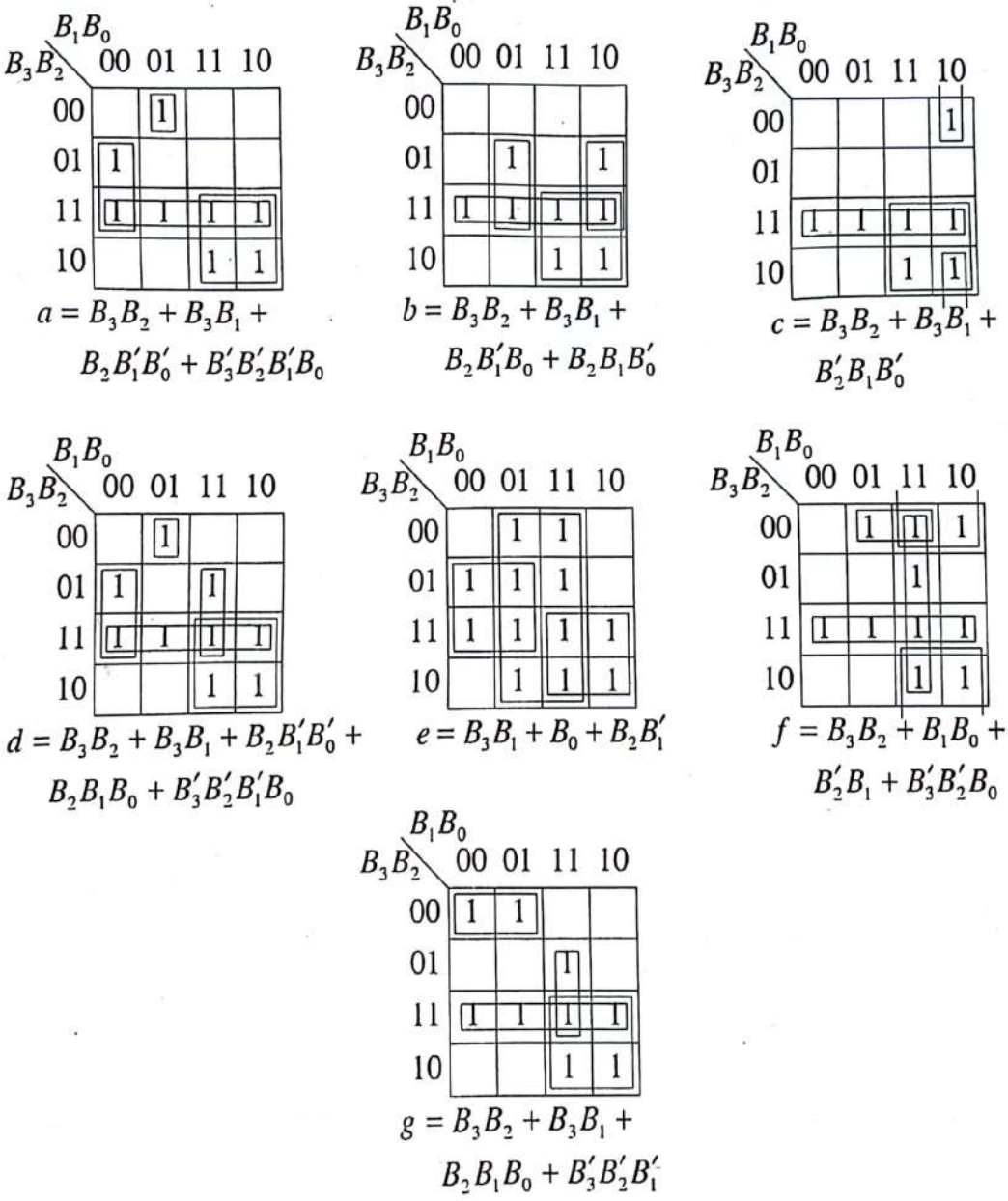


Figure 8.20 K-maps for BCD-to-7-segment display decoder with active-LOW output.

The logic implementation of the BCD-to-7-segment display decoder with active-LOW output is shown in Figure 8.21. Note that B_3B_2 is common to all outputs but e; B_3B_1 is common to all outputs but f; $B_3'B_2'B_1'B_0$ is common to a and d outputs; and $B_2B_1B_0$ is common to d and g outputs; $B_2B_1'B_0$ is common to a and d outputs.

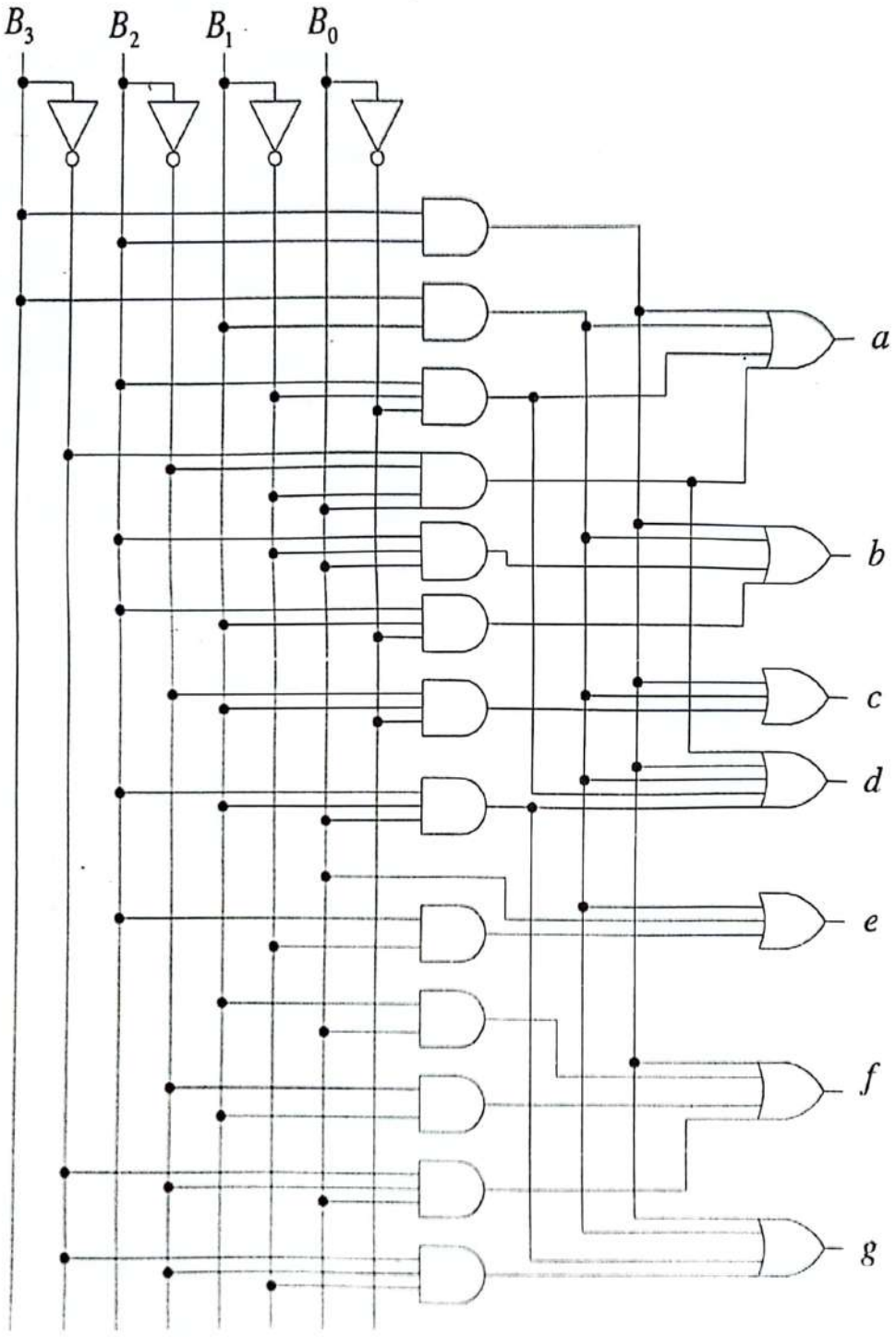


Figure 8.21 Logic implementation for BCD-to-7-segment display decoder with active-LOW output.

8.5.2 Available MSI BCD-to-7-Segment Display Decoder/Driver ICs

Available MSI BCD-to-7-segment display decoder/driver ICs are given in Table 8.3. Note that these MSI ICs also have LED driving capabilities.

Table 8.3 Available MSI BCD-to-7-segment display decoder/driver ICs.

IC No	Output
7446, 74246	Active-LOW; Open collector
7447, 74247	Active-LOW; Open collector
7448, 74248	Active-HIGH; Pull-up resistor = 2kΩ
7449, 74249	Active-HIGH; Open collector

IC 7447 is introduced here. The pin assignment of IC 7447 is shown in Figure 8.22. The IC 7447 accepts four lines of BCD input data ($B_0 - B_3$) and generates the corresponding seven-segment code with active-LOW, open collector outputs ($a' - g'$). If the input data is decimal zero, $RBI' = 0$ blanks the display. $BI / RBO' = 0$ unconditionally turns off all segment outputs. $LT' = 0$ turns on all segment outputs, provided that $BI / RBO' = 1$.

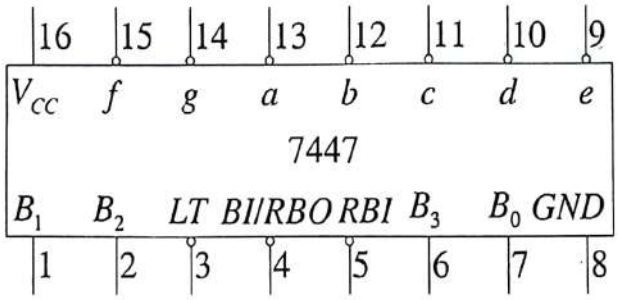


Figure 8.22 IC type 7447 BCD-to-7-segment display decoder/driver with active-LOW, open collector output.

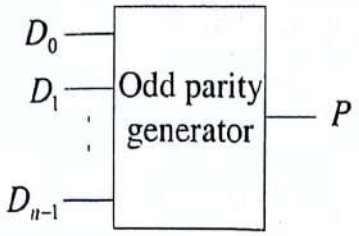
8.6 Parity Generator and Checker

8.6.1 Design of Parity Generator and Checker

As discussed in Chapter 2, a parity bit is an encoding scheme for detecting errors during transmission of binary information. A parity bit is an extra bit included with a binary message to make the number of 1s either odd (odd parity) or even (even parity). The message, including the parity bit, is transmitted and then checked at the receiving end for errors. An error is detected if the checked parity does not correspond to the one transmitted. The circuit that generates the parity bit in the transmitter is called a *parity generator*; the circuit that checks the parity in the receiver is called a *parity checker*.

Odd Parity Generator

Block diagram of an n-bit odd parity generator and its truth table for 3-bit and 4-bit messages are shown in Figure 8.23.



(a) Block diagram

D_2	D_1	D_0	P
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

(b) Truth table for 3-bit message

D_3	D_2	D_1	D_0	P
0	0	0	0	1
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	1
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

(c) Truth table for 4-bit message

Figure 8.23 Block diagram and truth table of odd parity generator.

From the truth table of Figure 8.23(b), we see that the function for the output P consists of four minterms with numerical values having an even number of 1s. This is the complement of EXOR function of the inputs. Remember that for an odd number of inputs EXOR function and EXNOR function are equal. So, it can also be said that the function for the output P is the complement of EXNOR function of the inputs. From this observation, the function for the output P can be written as

$$P = (D_2 \oplus D_1 \oplus D_0)'$$

From the truth table of Figure 8.23(c), we see that the function for the output P consists of eight minterms with numerical values having an even number of 0s. This is the EXNOR function of the inputs. Remember that for an even number of inputs EXOR function and EXNOR function are complement of each other. So, it can also be said that the function for the output P is the complement of EXOR function of the inputs. From this observation, the function for the output P can be written as

$$P = (D_3 \oplus D_2 \oplus D_1 \oplus D_0)'$$

From the above observations, we see that the function for the output P for an n number of inputs is the complement of the EXOR function of the inputs, that is,

$$P = (D_{n-1} \oplus D_{n-2} \oplus \cdots \oplus D_1 \oplus D_0)'$$

Logic implementation of an odd parity generator for 4-bit message is shown in Figure 8.24.

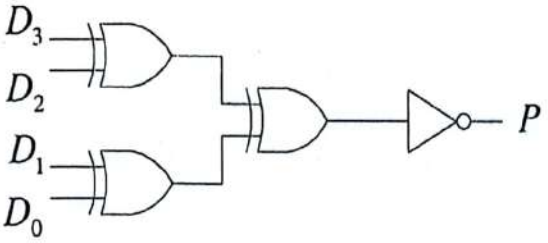


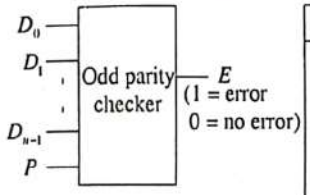
Figure 8.24 Logic implementation for odd parity generator for 4-bit message.

Odd Parity Checker

Block diagram of an odd parity checker for n-bit message ($n-1$ bit data plus a parity bit) and its truth table for 3-bit message and 4-bit message are shown in Figure 8.25. The truth tables of Figure 8.25(b) and (c) are similar to the truth tables of Figure 8.23(b) and (c). Following the same reasoning, the function for the output E of an odd parity checker for an n-bit message can be written as

$$E = (P \oplus D_{n-1} \oplus \cdots \oplus D_1 \oplus D_0)'$$

Logic implementation of an odd parity checker for 4-bit message is shown in Figure 8.26. From Figure 8.24 and Figure 8.26, we see that the same circuit can be used as both parity generator and parity checker.



(a) Block diagram

P	D_1	D_0	E
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

(b) Truth table for 3-bit message

P	D_2	D_1	D_0	E
0	0	0	0	1
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	1
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

(c) Truth table for 4-bit message

Figure 8.25 Block diagram and truth table of odd parity checker.

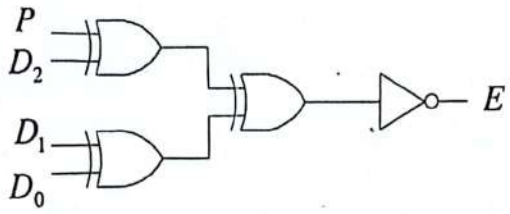


Figure 8.26 Logic implementation for odd parity checker for 4-bit message.

8.6.2 Available MSI Parity Generator/Checker IC

MSI IC type 74180 is an 8-bit parity generator/checker, which generates and checks both the odd and even parity. Pin assignment of IC 74180 is shown in Figure 8.27. Even and odd parity enable inputs are P_E and P_O respectively. Data inputs are $D_0 - D_7$ and even and odd parity outputs are E_E and E_O respectively.

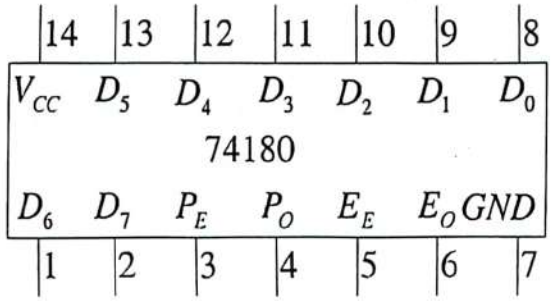


Figure 8.27 IC type 74180 8-bit parity generator/checker.

True active-HIGH or true active-LOW parity can be generated at both E_E and E_O outputs. True active-HIGH parity is established with enable inputs $P_E = 1$ and $P_O = 0$. True active-LOW parity is established when enable inputs $P_E = 0$ and $P_O = 1$.

Parity checking of a 9-bit word (8 bits plus parity) is possible by using the two enable inputs plus an inverter as the ninth data input. To check for true active-HIGH parity, the ninth data input is tied to the P_O input and an inverter is connected between the P_O and P_E inputs. To check for true active-LOW parity, the ninth data input is tied to the P_E input and an inverter is connected between the P_E and P_O inputs.

8.7 Multiplexers

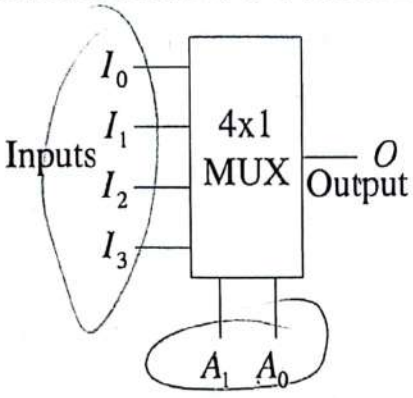
8.7.1 Design of Multiplexers

A digital multiplexer (MUX in short) is a combinational circuit that selects binary information from one of many input lines and directs it to a single output line. The selection of a particular input line is controlled by a set of address lines (also called selection lines). Normally, there are 2^n input lines and n address lines whose bit combinations determine which input is selected.

The block diagram and truth table of a 4-to-1-line multiplexer is shown in Figure 8.28. From the truth table of Figure 8.28(b), the Boolean function for the output O can be written as

$$O = I_0 A_1' A_0' + I_1 A_1' A_0 + I_2 A_1 A_0' + I_3 A_1 A_0$$

The logic implementation for the 4-to-1-line multiplexer is shown in Figure8.29.



(a) Block diagram

A_1	A_0	O
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

(b) Truth table

Figure 8.28 Block diagram and truth table of a 4-to-1-line multiplexer.

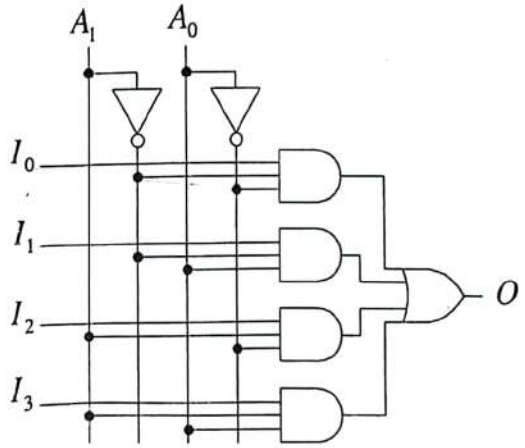


Figure 8.29 Logic implementation for a 4-to-1-line multiplexer.

A multiplexer is also called a data selector, since it selects one of many inputs and steers the binary information to the output line.

The AND gate portion of the multiplexer resembles a decoder circuit and, indeed, they decode the input address lines. In general, a 2^n-to-1-line multiplexer is constructed from an n-to-2^n-line decoder by adding to it 2^n input lines, one to each AND gate. The outputs of the AND gates are applied to a single OR gate to provide the 1-line output. The size of a multiplexer is specified by the number of 2^n of its input lines and the single output line. It is then implied that it also contains n address lines.

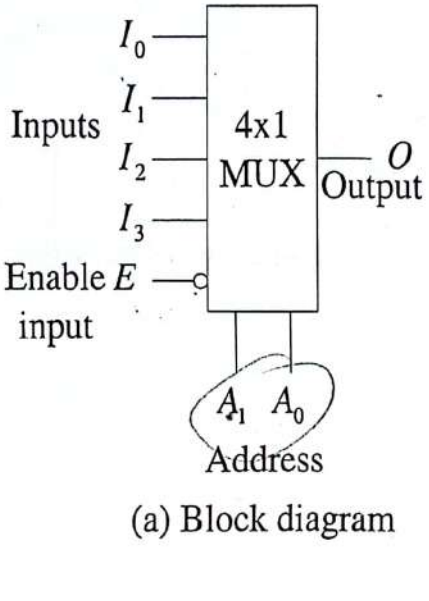
8.7.2 Multiplexer with Enable Input

Multiplexers may have an enable input to control the operation of the circuit. When the enable input is in active state, the circuit functions as a normal multiplexer, and when it is in inactive state, the outputs are disabled. Normally, the enable input is active-LOW.

The block diagram and truth table of a 4-to-1-line multiplexer with active-LOW enable input is given in Figure 8.30.From the truth table of Figure 8.30(b), the Boolean function for the output O can be written as

$$O = I_0E'A_1'A_0' + I_1E'A_1'A_0 + I_2E'A_1A_0' + I_3E'A_1A_0$$

The logic implementation for the 4-to-1-line multiplexer is shown in Figure 8.31.



(a) Block diagram

E	A_1	A_0	O
0	0	0	I_0
0	0	1	I_1
0	1	0	I_2
0	1	1	I_3
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

(b) Truth table

Figure 8.30 Block diagram and truth table of a 4-to-1-line multiplexer with active-LOW enable input.

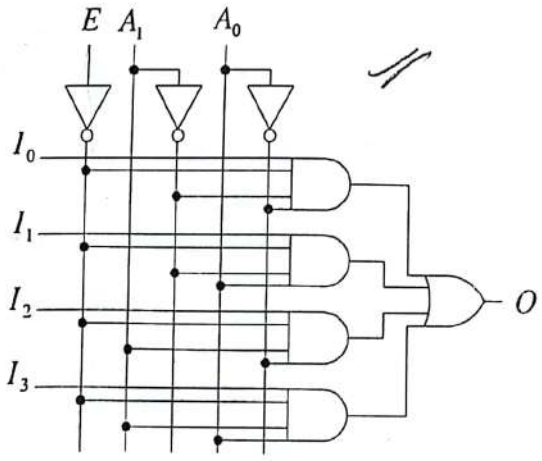


Figure 8.31 Logic implementation of a 4-to-1-line multiplexer with active-LOW enable input.

8.7.3 Available MSI Multiplexer ICs

Available MSI multiplexer ICs are given in Table 8.4.

Table 8.4 Available MSI multiplexer ICs.

IC No	Description	Output
74157	Quad 2×1 multiplexer	Same as input
74158	Quad 2×1 multiplexer	Inverted input
74153	Dual 4×1 multiplexer	Same as input
74352	Dual 4×1 multiplexer	Inverted input
74151	8×1 multiplexer	Complementary outputs
74150	16×1 multiplexer	Inverted input

IC type 74151 is illustrated here. IC 74151 is an 8-to-1-line MUX with active-LOW enable input. The pin assignment of the IC is shown in Figure 8.32. The eight inputs are designated I_0 through I_7. The three address or select lines $A_2A_1A_0$ select the particular input to be multiplexed and applied to the output. The IC provides both uncomplemented (O) and complemented (O') output. The IC also has an active-LOW enable input E.

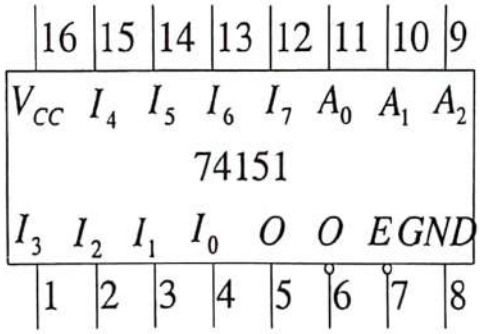


Figure 8.32 IC 74151 8-to-1-line multiplexer with active-LOW enable input.

8.7.4 Expansion of Muliplexer

To meet the larger inputs needs, multiplexers can be expanded by using enable input. Figure 8.33(a) shows an 8-to-1-line multiplexer and Figure 8.33(b) shows a 16-to-1-line multiplexer using 4-to-1-line multiplexers.

In Figure 8.33(a), when $A_2 = 0$, the multiplexer $M1$ is enabled and the multiplexer $M2$ is disabled. The output of the multiplexer $M2$ is 0 and the output of the multiplexer $M1$ is equal to its input corresponding to its A_1A_0 address bits. So, the final output of the OR gate is equal to the selected input of the multiplexer $M1$. When $A_2 = 1$, the multiplexer $M1$ is disabled and the multiplexer $M2$ is enabled. The output of the multiplexer $M1$ is 0 and the output of the multiplexer $M2$ is equal to its input corresponding to its A_1A_0 address bits. So, the final output of the OR gate is equal to the selected input of the multiplexer $M2$. For example, if $A_2A_1A_0 = 101$, that is $A_2 = 1$ and $A_1A_0 = 01$, then the multiplexer $M2$ will be

enabled and its I_1 input, which is I_5 input of the combined multiplexer, is selected and the final output will be equal to this input.

In Figure 8.33(b), when higher-order address bits $A_3A_2 = 00$, the output of the multiplexer *M5* is equal to the output of the multiplexer *M1*. The output of the multiplexer *M1* is equal to its input corresponding to the low-order address bits A_1A_0. Thus, the output of the multiplexer *M5* is equal to the input of the multiplexer *M1* corresponding to A_1A_0. Similarly, when $A_3A_2 = 01$, the output of the multiplexer *M5* is equal to the input of the multiplexer *M2* corresponding to A_1A_0. When $A_3A_2 = 10$, the output of the multiplexer *M5* is equal to the input of the multiplexer *M3* corresponding to A_1A_0. Finally, when $A_3A_2 = 11$, the output of the multiplexer *M5* is equal to the input of the multiplexer *M4* corresponding to A_1A_0. For example, when $A_3A_2A_1A_0 = 0110$, that is $A_3A_2 = 01$ and $A_1A_0 = 10$, the output of the multiplexer *M5* will be equal to its I_1 input. The input I_1 of the multiplexer *M5* will be equal to the output of the multiplexer *M2*. The output of the multiplexer *M2* will be equal to its input I_2, which is the input I_6 of the combined multiplexer.

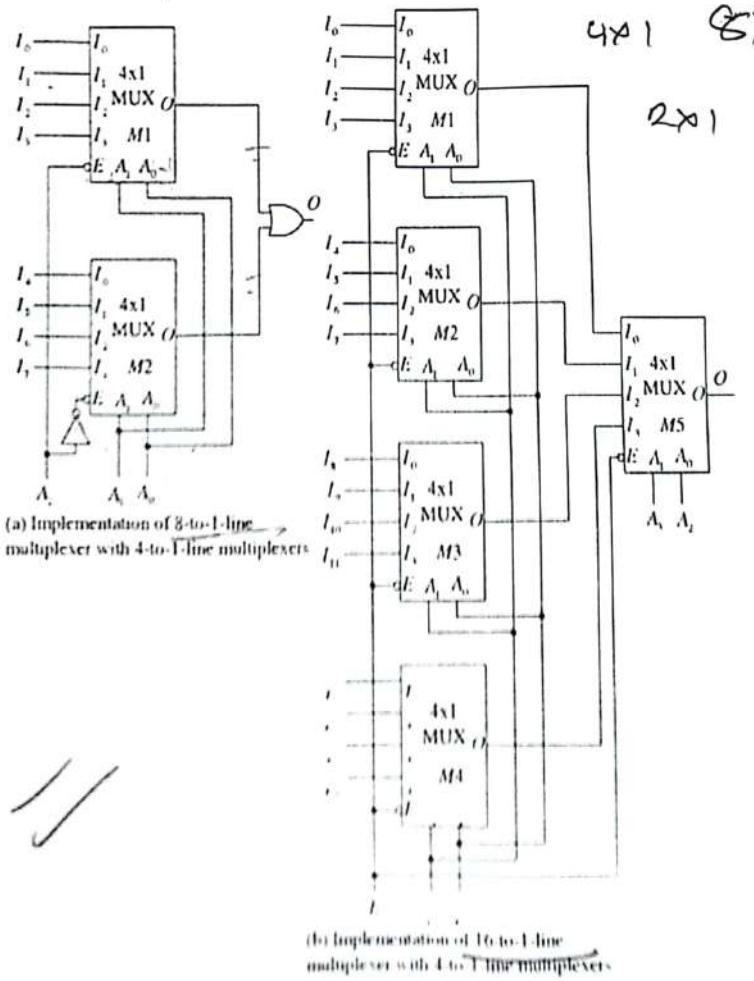


Figure 8.33 Expansion of multiplexers.

8.7.5 Combinational Logic Implementation with Multiplexers

It was shown in Section 8.3 that a decoder with active-HIGH output could be used to implement a Boolean function by employing an external OR gate. A quick reference to the multiplexer of Figure 8.29 reveals that it is essentially a decoder with active-HIGH output with OR gate already available. The minterms out of the decoder to be chosen can be controlled with the input lines. The minterm to be included with the function being implemented are chosen by making their corresponding input lines equal to 1, those minterms not included in the function are disabled by making their input lines equal to 0. The input variables of the function are applied to the address lines. This gives a method for implementing any Boolean function of n variables with a 2^n-to-1-line multiplexer. The implementation of the function

$$F(A,B,C,D) = \sum(0,2,3,6,8,9,12,14)$$

using a 16×1 multiplexer is shown in Figure 8.34.

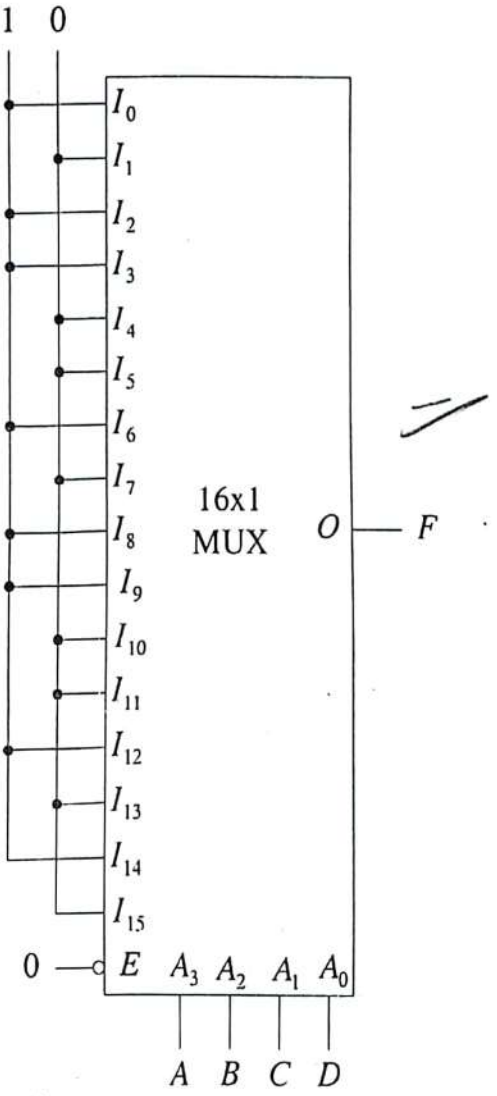


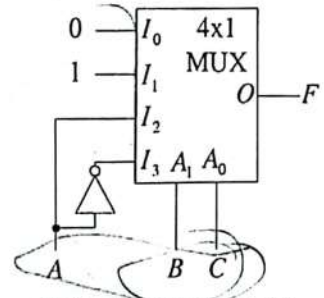
Figure 8.34 Implementation of the function $F(A,B,C,D) = \sum(0,2,3,6,8,9,12,14)$ using 16×1 multiplexer.

It is possible to do better than the above procedure. If we have a Boolean function of $n+1$ variables, we take n of these variables and connect them to the address lines of a multiplexer. The remaining single variable of the function is used for the inputs of the multiplexer. If A is this single variable, the inputs of the multiplexer are chosen to be either A or A' or 1 or 0. By judicious use of these four values for the inputs and by connecting the other variables to the address lines, we can implement any Boolean function with a multiplexer. In this way, it is possible to generate any function of $n+1$ variables with a 2^n-to-1-line multiplexer.

To demonstrate this procedure with a concrete example, consider the following function of three variables

$$F(A,B,C) = \sum(1,3,5,6)$$

The function can be implemented with a 4-to-1-line multiplexer as shown in Figure 8.35. Two of the variables, B and C are applied to the address lines in that order, i.e., B is connected to A_1 and C to A_0. The inputs of the multiplexer are 0, 1, A, and A'. When $BC = 00$, output $F = 0$ since $I_0 = 0$. Therefore, both minterms $m_0 = A'B'C'$ and $m_4 = AB'C'$ produce a 0 output, since the output is 0 when $BC = 00$ regardless of the value of A. When $BC = 01$, output $F = 1$, since $I_1 = 1$. Therefore, both minterms $m_1 = A'B'C$ and $m_5 = AB'C$ produce a 1 output, since the output is 1 when $BC = 01$ regardless of the value of A. When $BC = 10$, input I_2 is selected. Since A is connected to this input, the output will be equal to 1 only for minterm $m_6 = ABC'$, but not for minterm $m_2 = A'BC'$, because when $A' = 1$, then $A = 0$, and since $I_2 = 0$, we have $F = 0$. Finally, when $BC = 11$, input I_3 is selected. Since A' is connected to this input, the output will be equal to 1 only for minterm $m_3 = A'BC$, but not for $m_7 = ABC$. This information is summarized in Figure 8.35(b), which is the truth table of the function we want to implement.



(a) Implementation with multiplexer

Minterm	A	B	C	F
0	0	0	0	0
1	0	0	1	1
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	1
7	1	1	1	0

(b) Truth table

	I_0	I_1	I_2	I_3
A'	0	①	2	③
A	4	⑤	⑥	7
	0	1	A	A'

(c) Implementation table

Figure 8.35 Implementation of the function $F(A,B,C) = \sum(1,3,5,6)$ with a multiplexer.

The above discussion shows by analysis that the multiplexer implements the required function. We now present a general procedure for implementing any Boolean function of n variables with a 2^{n-1}-to-1 multiplexer.

First, express the function in its sum of minterms form. Assume that the ordered sequence of variables chosen for the minterms is $ABCD\cdots$, where A is the leftmost variable in the ordered sequence of n variables and $BCD\cdots$ are the remaining $n-1$ variables. Connect the $n-1$ variables to the address lines of the multiplexer with B connected to the high-order address line, C to the next lower address line, and so on down to the last variable, which is connected to the lowest-order line A_0. Consider now the single variable A. Since this variable is in the highest-order position in the sequence of variables, it will be complemented in minterms 0 to $(2^n/2)-1$ which comprise the first half in the list of minterms. The second half of the minterms will have their A variable uncomplemented. For a three-variable function, A, B, C, we have eight minterms. Variable A is complemented in minterms 0 to 3 and uncomplemented in minterms 4 to 7.

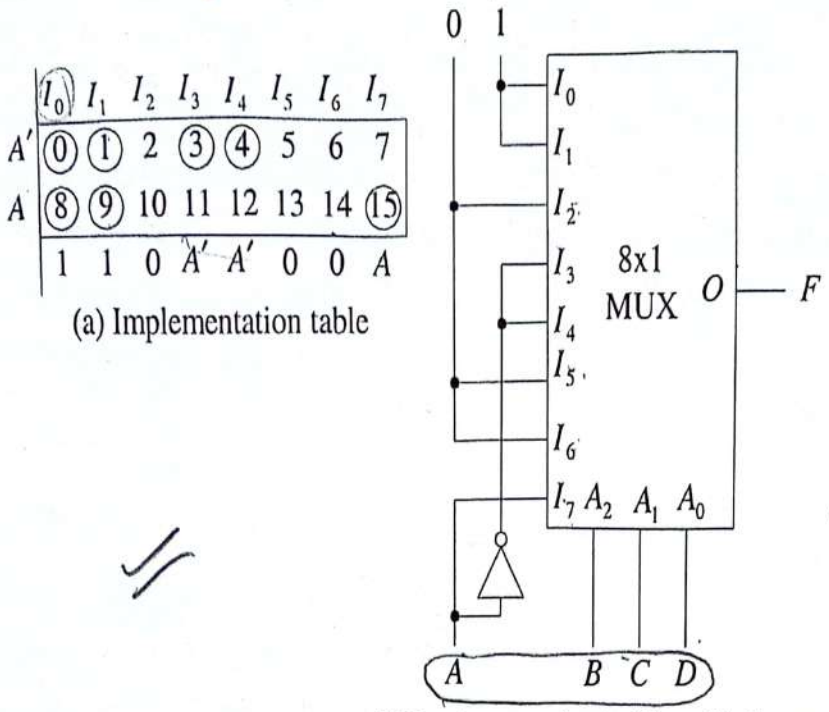
List the inputs of the multiplexer and under them list all the minterms in two rows. The first row lists all those minterms where A is complemented, and the second row all the minterms with A uncomplemented, as shown in Figure 8.35(c). Circle all the minterms of the function and inspect each column separately. If the two minterms in a column are not circled, apply 0 to the corresponding multiplexer input. If the two minterms are circled, apply 1 to the corresponding multiplexer input. If the bottom minterm is circled and the top is not circled, apply A to the corresponding multiplexer input. If the top minterm is circled and the bottom is not circled, apply A' to the corresponding multiplexer input. This procedure follows the condition established during the previous analysis.

Figure 8.35(c) shows the implementation table for the Boolean function $F(A,B,C)=\sum(1,3,5,6)$. From which we obtain the multiplexer connections of Figure 8.35(a). Note that B must be connected to A_1 and C to A_0.

The procedure is illustrated with another example. Consider the function

$$F(A,B,C,D)=\sum(0,1,3,4,8,9,15)$$

This is a four-variable function and therefore we need a multiplexer with three address lines and eight inputs. We apply variables B, C, and D to the address lines. The implementation table is shown in Figure 8.36(a). The first half of the minterms are associated with A' and the second half with A. By circling the minterms of the function and applying the rules for finding values for the multiplexer inputs, we obtain the implementation as shown in Figure 8.36(b).



(a) Implementation table

(b) Implementation with multiplexer

Figure 8.36 Implementation of the function $F(A,B,C,D) = \sum(0,1,3,4,8,9,15)$ with multiplexer.

Let us now compare the multiplexer method with the decoder method for implementing combinational circuits. The decoder method requires an external gate for each output function, but only one decoder is needed to generate all output functions. The multiplexer method uses smaller-size units but requires one multiplexer for each output function. It would seem reasonable to assume that combinational circuits with a small number of outputs should be implemented with multiplexers. Combinational circuits with many output functions would probably use fewer ICs with the decoder method.

8.8 Demultiplexers

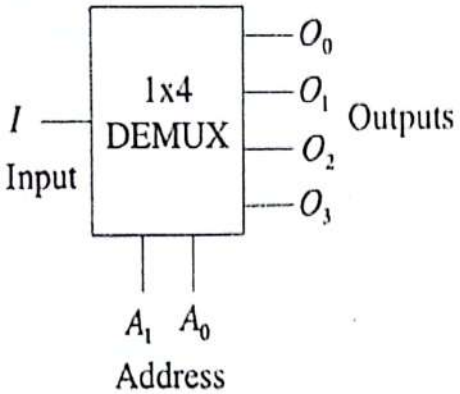
8.8.1 Design of Demultiplexers

A demultiplexer (DEMUX in short) is a combinational circuit that receives information on a single line and transmits this information on one of 2^n possible output lines. The selection of a specific output line is controlled by n address lines.

The block diagram and truth table of a 1-to-4-line demultiplexer is shown in Figure 8.37. From the truth table of Figure 8.37(b), the Boolean functions for the outputs can be written as

$$O_0 = IA_1'A_0'$$
$$O_1 = IA_1'A_0$$
$$O_2 = IA_1A_0'$$
$$O_3 = IA_1A_0$$

The logic implementation of a 1-to-4-line demultiplexer is shown in Figure 8.38.



(a) Block diagram

I	A_1	A_0	O_3	O_2	O_1	O_0
0	0	0	0	0	0	0
0	0	1	0	0	0	0
0	1	0	0	0	0	0
0	1	1	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

(b) Truth table

Figure 8.37 Block diagram and truth table of 1-to-4-line demultiplexer.

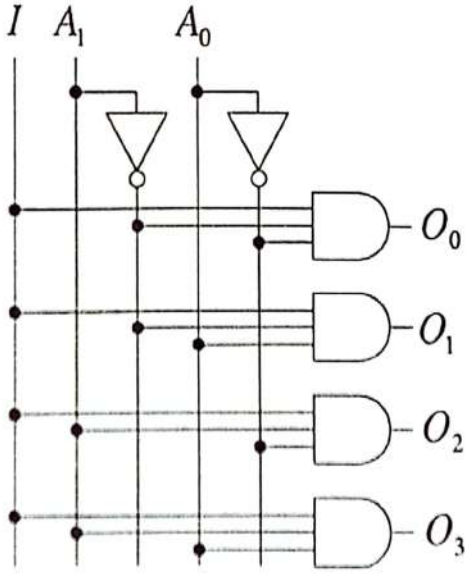


Figure 8.38 Logic implementation for 1-to-4-line demultiplexer.

Note that the logic circuit of Figure 8.38 is indeed a 2-to-4-line decoder with active-HIGH output and active-HIGH enable, where the input line I is the enable input line. Also note that the logic circuit of a 2-to-4-line decoder with active-LOW output and active-LOW enable input, as shown in Figure 8.9, can be used as an 1-to-4-line demultiplexer if the enable input E is taken as the input line. This can be verified from the truth table of Figure 8.8(b) that the E input appears at the output selected by the address line A_1A_0. The only difference is that the values of the unselected lines remain 1

8.8.2 Available MSI Demultiplexer ICs

Because decoder and demultiplexer operations are obtained from the same circuit, some MSI ICs are referred to as *decoder/demultiplexer*. List of available MSI demultiplexer ICs are given in Table 8.5. Note that the same ICs are also listed in Table 8.2 as decoders.

Table 8.5 Available MSI demultiplexer ICs.

IC No	Description	Output
74139	Dual 1×4 Demultiplexer	Same as input
74155	Dual 1×4 Demultiplexer	Inverted input/Same as input
74156	Dual 1×4 Demultiplexer	Inverted input/Same as input (Open-collector)
74138	1×8 Demultiplexer	Inverted input/Same as input
74154	1×16 Demultiplexer	Same as input

8.8.3 Expansion of Demultiplexer

As in decoders, to meet the larger output needs, demultiplexers can be expanded as shown in Figure 8.39, which shows an 1-to-16-line demultiplexer using 1-to-4-line demultiplexer.

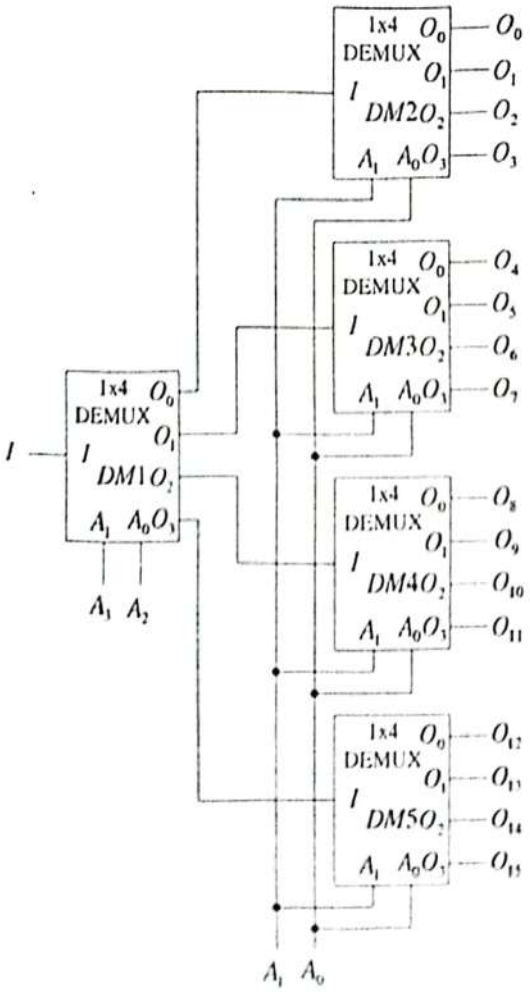


Figure 8.39 Expansion of demultiplexers.

In Figure 8.39, when high-order address bits $A_3A_2 = 00$, the external input I appears at the O_0 output of the demultiplexer $DM1$, which is the input to the demultiplexer $DM2$. Then this input will appear at the output of the demultiplexer $DM2$ which is selected by the low-order address bits A_1A_0. When $A_3A_2 = 01$, the external input I will appear at the O_1 output of the demultiplexer $DM1$, which will then appear at the output of the demultiplexer $DM3$ selected by A_1A_0. When $A_3A_2 = 10$, the external input I will appear at the O_2 output of the demultiplexer $DM1$, which will then appear at the output of the demultiplexer $DM4$ selected by A_1A_0. Finally, when $A_3A_2 = 11$, the external input I will appear at the O_3 output of the demultiplexer $DM1$, which will then appear at the output of the demultiplexer $DM5$ selected by A_1A_0. For example, when $A_3A_2A_1A_0 = 0110$, that is $A_3A_2 = 01$ and $A_1A_0 = 10$, the external input I will appear at the O_1 output of the demultiplexer $DM1$ as $A_3A_2 = 01$. This output is connected to the input of the demultiplexer $DM3$. Then this input to the demultiplexer $DM3$ will appear at its O_2 output as $A_1A_0 = 10$. This output is the output O_6 of the combined demultiplexer.

8.8.4 Combinational Logic Implementation with Demultiplexers

When the input $I = 1$, a demultiplexer functions as a decoder with active-HIGH output. Then combinational logic implementation using demultiplexers becomes similar to combinational logic implementation using decoders. Implementation of the following 3-variable functions using a 1-to-8-line demultiplexer is shown in Figure 8.40.

$$F_1(A,B,C) = \sum(1,2,4,7)$$
$$F_2(A,B,C) = \sum(0,5,6,7)$$

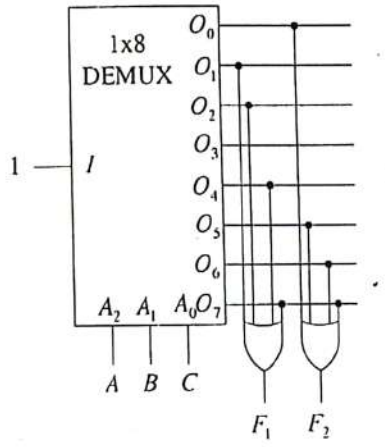


Figure 8.40 Implementation of the functions $F_1(A,B,C) = \sum(1,2,4,7)$ and $F_2(A,B,C) = \sum(0,5,6,7)$ using demultiplexer.

Exercises

8.1 Design the following encoders for both active-LOW and active-HIGH inputs.
(a) Decimal-to-BCD encoder.
(b) Decimal-to-Excess-3 encoder.
(c) Hexadecimal-to-binary encoder.
(d) 12-4-line encoder.

8.2 Repeat Problem 8.1 for inverted output.

8.3 Design the following decoders for both active-LOW and active-HIGH output.
(a) 4-to-16-line decoder.
(b) 4-to-12-line decoder.

8.4 Repeat Problem 8.3 with active-LOW enable input.

8.5 (a) Design a BCD-to-decimal decoder with active-HIGH output.
(b) Design an Excess-3-to-decimal decoder with both active-LOW and active-HIGH output.

8.6 Repeat Problem 8.5 with active-LOW enable input.

8.7 Design a 1-of-40-line decoder with active-LOW output using BCD-to-decimal decoders with active-LOW output.

8.8 Construct a 5×32 decoder with active-LOW output using four 3×8 decoders with active-LOW output and a 2×4 decoder with active-LOW output.

8.9 Implement a full-adder circuit using decoders with both active-HIGH and active-LOW output.

8.10 Implement a full-subtractor circuit using decoders with both active-HIGH and active-LOW output.

8.11 Design a BCD-to-Excess-3 code converter using a BCD-to-decimal decoder with active-HIGH output and four OR gates.

8.12 A combinational circuit is defined by the following three functions:

$$F_1 = A'B' + ABC'$$
$$F_2 = A' + B$$
$$F_3 = AB + A'B'$$

Design the circuit with a decoder with both active-LOW and active-HIGH output and external gates.

8.13 A combinational circuit is defined by the following two functions.

$$F_1(A,B) = \sum(0,3)$$
$$F_2(A,B) = \sum(1,2,3)$$

Implement the combinational circuit by means of a decoder with active-HIGH output and external NAND gate.

8.14 Design an Excess-3-to-BCD code converter using classical method.

8.15 Design an Excess-3-to-BCD code converter using 4-bit binary adder.

8.16 Design a BCD-to-7-segment display decoder with active-HIGH output.

8.17 Design a BCD-to-7-segment display decoder with both active-LOW and active-HIGH output using BCD-to-decimal decoder with active-LOW and active-HIGH output respectively and external gates.

8.18 Design a Hexadecimal-to-7-segment display decoder with both active-LOW and active-HIGH output.

8.19 Design a BCD-to-7-segment display decoder with active-LOW output using 1-to-16-line decoder with active-LOW output and external gates.

8.20 Design a BCD-to-7-segment display decoder with active-LOW output using a BCD-to-decimal decoder with active-LOW output and external gates.

8.21 Draw a logic diagram for odd parity generator/checker for 3-bit message.

8.22 Design an even parity generator/checker for n-bit message.

8.23 Construct a 4-to-1-line multiplexer using a 2-to-4-line decoder with active-HIGH output and external AND and OR gates.

8.24 Design a 4-to-1-line multiplexer with active-HIGH enable input.

8.25 Construct an 8×1 multiplexer with two 4×1 multiplexers having active-LOW enable inputs.

8.26 Construct a 32×1 multiplexer with four 8×1 multiplexers and one 4×1 multiplexer.

8.27 Construct a 32×1 multiplexer using two 16×1 multiplexers.

8.28 Implement the Boolean function

$$F(A,B,C,D) = \sum(0,1,3,4,8,9,15)$$

using a 16×1 multiplexer.

8.29 Implement the following functions with two 4×1 multiplexers, an OR gate, and inverter.

$$F_1 = x'y' + xyz'$$

$$F_2 = x' + y$$

$$F_3 = xy + x'y'$$

8.30 Realize the function

$$F(A,B,C,D) = \sum(2,4,6,7,9,10,11,12,15)$$

using

(a) a 16×1 multiplexer

(b) an 8×1 multiplexer.

8.31 Implement a full-adder circuit with multiplexer.

8.32 Design a BCD-to-7-segment display decoder with active-LOW output using 4×1 multiplexers and external gates.

8.33 Design a 40×1 multiplexer using 8×1 multiplexers.

8.34 Construct an 1-to-32-line demultiplexer with four 1-to-8-line demultiplexers and one 1-to-4-line demultiplexers.

8.35 Design an 1× 40 demultiplexer using 1×8 demultiplexers.

Further Reading

1. *The TTL Data Book*. Dallas, Texas Instruments, 1988.
2. Richard S. Sandige, *Modern Digital Design*, McGraw-Hill Publishing Company, 1990.
3. R. P. Jain, *Modern Digital Electronics*, 2nd ed., Tata McGraw-Hill Publishing Company Limited, India, 1998.
4. *Microprocessor Data Hand Book*, Revised Ed. BPB Publications, India, 2000.
5. M. Morris Mano, *Digital Logic and Computer Design*, Prentice-Hall, Inc., 2000.
6. John P. Uyemura, *Digital Systems Design: An Integrated Approach*, Brooks/Cole Publishing Company, 2000.
7. D. C. Green, *Digital Electronics*, 5th ed., Pearson Education, 2001.
8. M. Morris Mano, *Digital Design*, 3rd ed., Pearson Education, 2002.